

Aligarh Muslim University

Department of Computer Science

LAB FILE

**Submitted By:**

Name: Sumaniya Sheikh

Enrolment Number: GR0960

Faculty Number: 25CAMSA144

Subject Code: CAMS3P01

Subject Name: Laboratory Course -III

Submitted To:

Dr. Faisal Anwer

Dr. Faraz Masood

Mr. Saad Shamim

Mr. Ahsan Shameem

Week #1

1. Write a program to find the product of two user-supplied integers and if the product is equal to or lower than 5000, then return the sum of the two numbers.

Solution:

```
num1,num2= int(input("enter a number = ")),int(input("enter a number = "));
print(num1+num2 if num1*num2<=5000 else prod)
```

```
Problems 1 Output Debug Console Terminal Ports Python + - [ ] [X] ... [ ] [X]
Warning: PowerShell detected that you might be using a screen reader and has disabled PSReadLine for compatibility purposes. If you want to re-enable it, run 'Import-Module PSReadLine'.

PS E:\3rd sem\Python-Lab-work> & "C:\Program Files\Python310\python.exe" "e:/3rd sem/Python-Lab-work/week1/Q1.py"
enter a number = 2
enter a number = 60
62
PS E:\3rd sem\Python-Lab-work> [ ]
```

- 2. Write a program to print the sum of the first 10 numbers**

Solution:

```
for i in range(1, 11):
    sum=sum+i
print(sum)
```

```
PS E:\3rd sem\Python-Lab-Work> & "C:\Program Files\Python310\python.exe" "e:/3rd sem/Python-Lab-Work/week1/q2.py"
1
3
6
10
15
21
28
36
45
55
PS E:\3rd sem\Python-Lab-Work>
```

- 3. Write a program to iterate through a supplied list of 20 numbers and print only those numbers which are divisible by 5.**

Solution:

```
num=[1,2,3,4,5,6,7,8,9,10,11,12,13,14,30,24,34,32,27,35]
for i in num:
    if i%5==0:
        print(i)
```

```
PS E:\3rd sem\Python-Lab-Work> & "C:\Program Files\Python310\python.exe" "e:/3rd sem/Python-La
b-Work/week1/Q3.py"
5
10
30
35
PS E:\3rd sem\Python-Lab-Work> █
```

4. Write a program to check if the given number is a palindrome.**Solution:**

```
num=(input("enter a number = "))
if str(num)==str(num)[::-1]:
    print("palindrome")
else:
    print("not palindrome")
```

```
PS E:\3rd sem\Python-Lab-Work> & "C:\Program Files\Python310\python.exe" "e:/3rd sem/Python-La
b-Work/week1/Q4.py"
enter a number = 242
palindrome
PS E:\3rd sem\Python-Lab-Work> & "C:\Program Files\Python310\python.exe" "e:/3rd sem/Python-La
b-Work/week1/Q4.py"
enter a number = 245
not palindrome
PS E:\3rd sem\Python-Lab-Work> █
```

5. Write a program to calculate the cube of all numbers from 1 to a given number.**Solution:**

```
num=int(input("enter a three digit number = "))  
for i in range(1, num):  
    cube=i*i*i  
    print(cube)
```

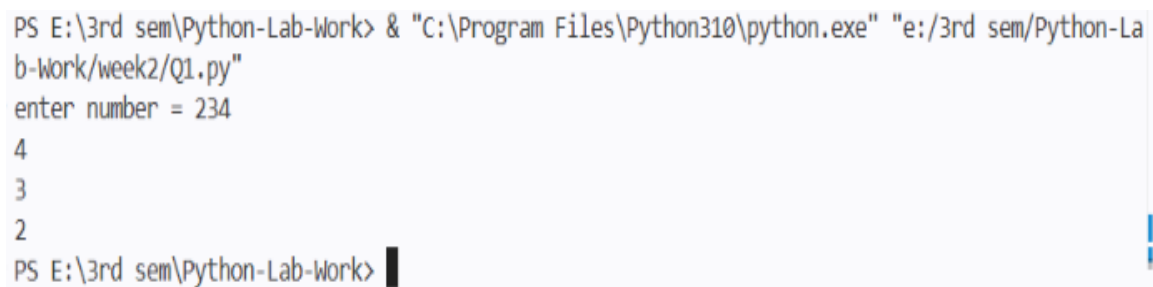
```
PS E:\3rd sem\Python-Lab-Work> & "C:\Program Files\Python310\python.exe" "e:/3rd sem/Python-Lab-Work/week1/Q5.py"  
enter a number = 5  
1  
8  
27  
64  
125  
PS E:\3rd sem\Python-Lab-Work> █
```

Week #2

1. Write a program to extract each digit from an integer in reverse order.

Solution:

```
num=int(input("enter number = "))
while num>0:
    digit= num%10
    print(digit)
    num=num//10
```



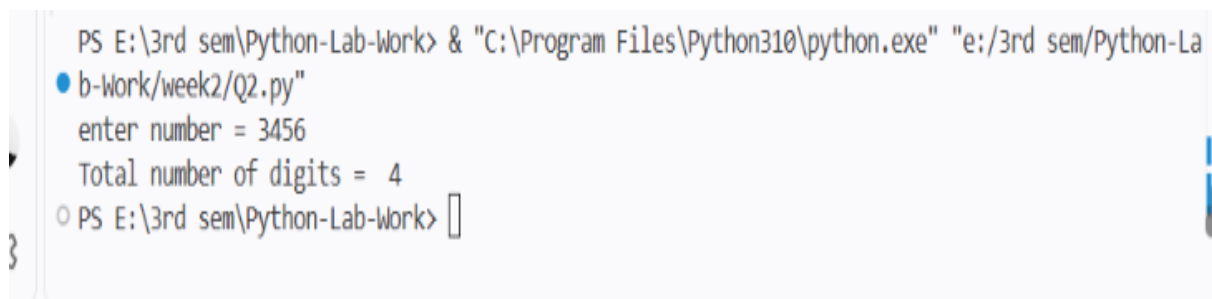
```
PS E:\3rd sem\Python-Lab-Work> & "C:\Program Files\Python310\python.exe" "e:/3rd sem/Python-Lab-Work/week2/Q1.py"
enter number = 234
4
3
2
PS E:\3rd sem\Python-Lab-Work> █
```

2. Write a program to count the total number of digits in a number using a while loop.

Solution:

```
num=int(input("enter number = "))
count=0
while num>0:
    digit= num%10
    count=count+1
    num=num//10

print("Total number of digits = ", count)
```



```
PS E:\3rd sem\Python-Lab-Work> & "C:\Program Files\Python310\python.exe" "e:/3rd sem/Python-Lab-Work/week2/Q2.py"
enter number = 3456
Total number of digits = 4
PS E:\3rd sem\Python-Lab-Work> █
```

3. Write a program to display all prime numbers within a range.

Solution:

```
start=int(input("enter the starting number = "))
end=int(input("enter the end range = "))
for num in range(start, end+1):
    if num<2:
        continue
    is_prime =True
    for i in range(2, int(num ** 0.5)+1):
        if num%i==0:
            is_prime=False
            break
    if is_prime:
        print(num)
```



The screenshot shows a Windows command prompt window with the following text:

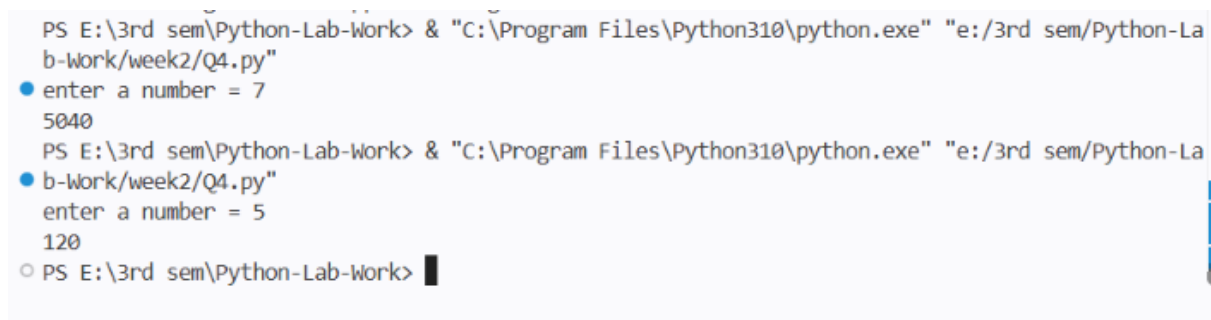
```
PS E:\3rd sem\Python-Lab-Work> & "C:\Program Files\Python310\python.exe" "e:/3rd sem/Python-Lab-Work/week2/Q3.py"
enter the starting number = 2
enter the end range = 6
prime number: 2
prime number: 3
prime number: 5
PS E:\3rd sem\Python-Lab-Work>
```

The command prompt window has a taskbar at the bottom with icons for main*, 01:11, 0/0, CRLF, Python, Python 3.10 (64-bit), Go Live, Prettier, and a bell icon.

4. Write a program to use a loop to find the factorial of a given number.

Solution:

```
from functools import reduce;
n=int(input("enter a number = ")); fact=(reduce(lambda x,y: x*y,
range(1,n+1),1))
print(fact)
```



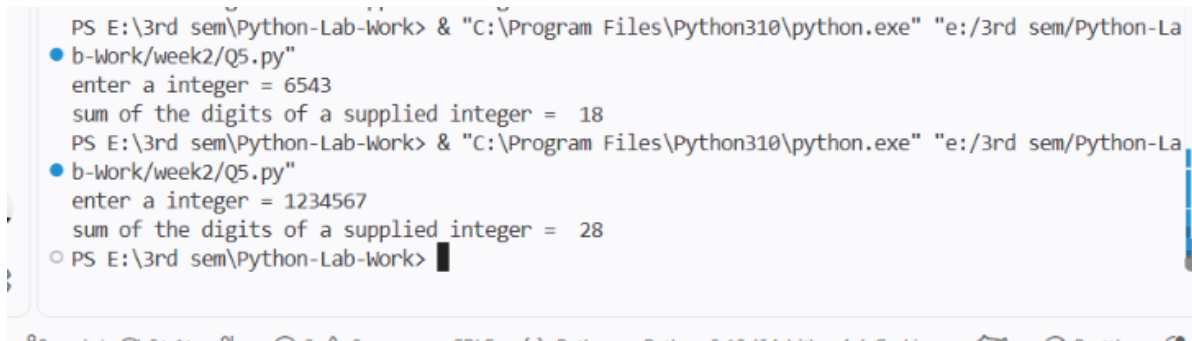
The screenshot shows a Windows command prompt window with the following text:

```
PS E:\3rd sem\Python-Lab-Work> & "C:\Program Files\Python310\python.exe" "e:/3rd sem/Python-Lab-Work/week2/Q4.py"
enter a number = 7
5040
PS E:\3rd sem\Python-Lab-Work> & "C:\Program Files\Python310\python.exe" "e:/3rd sem/Python-Lab-Work/week2/Q4.py"
enter a number = 5
120
PS E:\3rd sem\Python-Lab-Work>
```

5. Write a program to find the sum of the digits of a supplied integer.

Solution:

```
sum=sum(map(int,input("enter a integer = ")))  
print("sum of the digits of a supplied integer = ",sum)
```



```
PS E:\3rd sem\Python-Lab-Work> & "C:\Program Files\Python310\python.exe" "e:/3rd sem/Python-Lab-Work/week2/Q5.py"  
enter a integer = 6543  
sum of the digits of a supplied integer = 18  
PS E:\3rd sem\Python-Lab-Work> & "C:\Program Files\Python310\python.exe" "e:/3rd sem/Python-Lab-Work/week2/Q5.py"  
enter a integer = 1234567  
sum of the digits of a supplied integer = 28  
PS E:\3rd sem\Python-Lab-Work>
```

Week #3

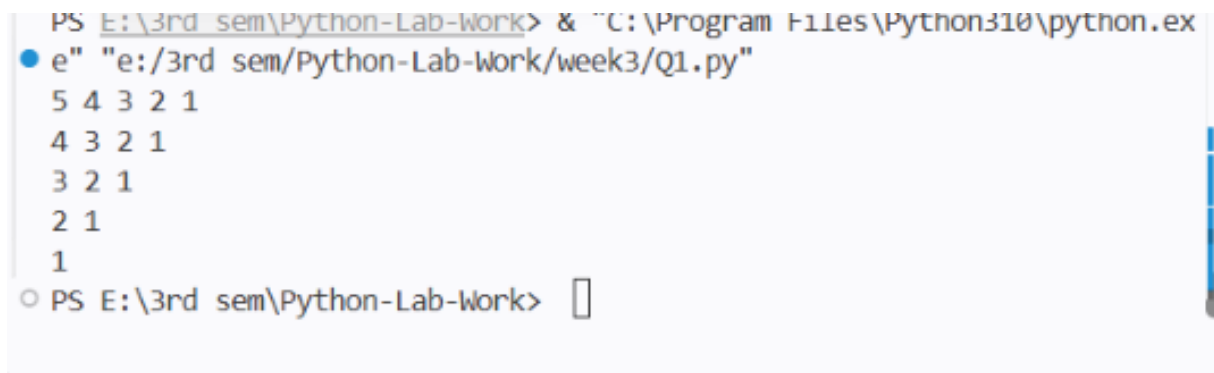
1. Write a program to print the following pattern using the for loop:

```
5 4 3 2 1
4 3 2 1
3 2 1
2 1
1
```

Solution:

```
# for i in range(5,0,-1):
#     for j in range(i,0,-1):
#         print(j, end=" ")
#     print()
```

```
for i in range(5,0,-1):
    print(range(i,0,-1))
```



```
PS E:\3rd sem\Python-Lab-work> & "C:\Program Files\Python310\python.exe"
e" "e:/3rd sem/Python-Lab-work/week3/Q1.py"
5 4 3 2 1
4 3 2 1
3 2 1
2 1
1
PS E:\3rd sem\Python-Lab-work>
```

2. Write a program to print the following star pattern using the for loop:

```
*
* *
* * *
* * * *
* * * * *
* * * *
* * *
* *
*
```

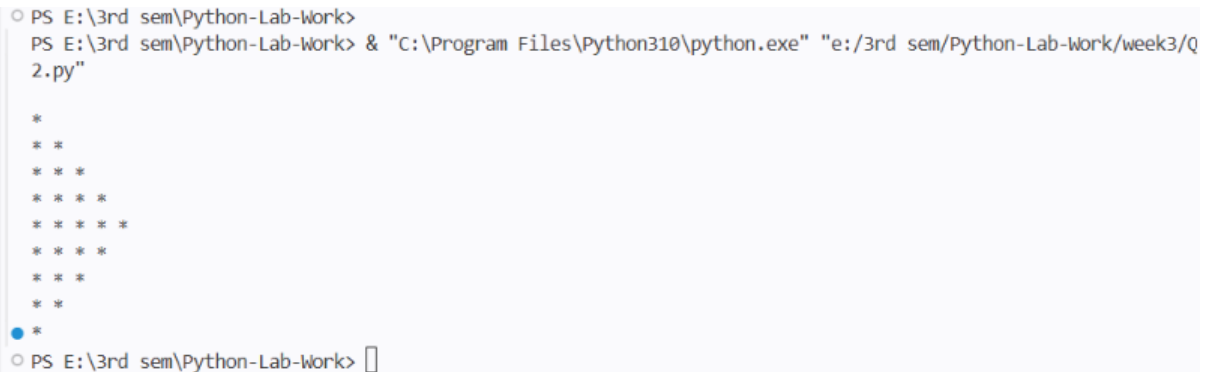
Solution:

```
for i in range(1, 10):
    print()
```

```

if i<=5:
    for j in range(1, i+1):
        print("*", end=" ")
else:
    for j in range(10-i):
        print("*", end=" ")

```



```

PS E:\3rd sem\Python-Lab-Work> & "C:\Program Files\Python310\python.exe" "e:/3rd sem/Python-Lab-Work/week3/Q2.py"
*
* *
* * *
* * * *
* * * * *
* * * * *
* * * *
* * *
* *
*
PS E:\3rd sem\Python-Lab-Work>

```

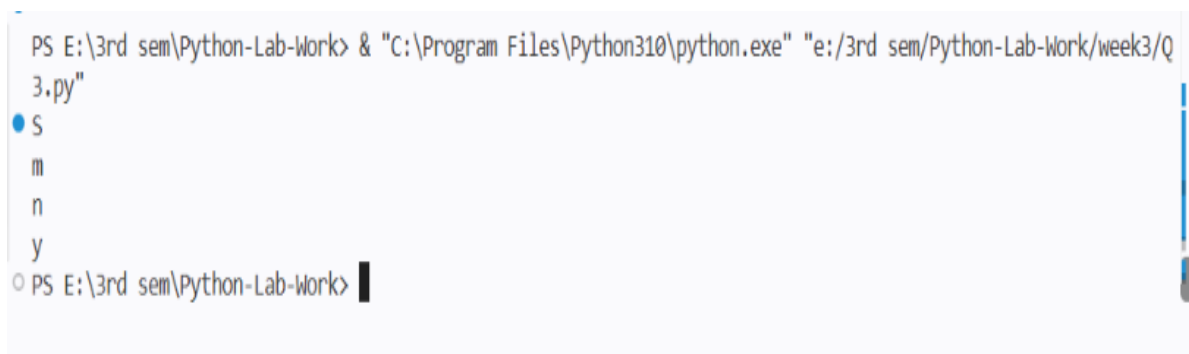
3. Write a program to print characters from a string which are present at even index numbers.

Solution:

```

str="Sumaniya"
for i in range(len(str)):
    if i%2==0:
        print(str[i])

```



```

PS E:\3rd sem\Python-Lab-Work> & "C:\Program Files\Python310\python.exe" "e:/3rd sem/Python-Lab-Work/week3/Q3.py"
S
m
n
y
PS E:\3rd sem\Python-Lab-Work>

```

4. Write a program to accept a string from the user and display characters that are present at even index numbers.

Solution:

```
str= input("enter a string = ")
for i in range(len(str)):
    if i%2==0:
        print(str[i])
```



5. Write a program to remove characters from a string starting from the nth position to the last and return a new string. Example: `remove_chars("aligarh", 3)` should output ali.

Solution:

```
def remove_chars(str,num):return str[:num]
str= input("enter a string = ")
num=int(input("enter the position = "))
print(remove_chars(str,num))
```



Week #4

1. Write a program to create a function `cal_sum_sub()` that accepts two variables and calculates addition and subtraction. Also, it must return both addition and subtraction in a single return call.

Solution:

```
def cal_sum_sub(a,b):  
    return a+b, a-b
```

```
a,b = map(int,input("enter first number = ").split())
```

```
print(cal_sum_sub(a,b))
```

```
PS E:\3rd sem\Python-Lab-Work> & "C:\Program Files\Python310\python.exe" "e:/3rd sem/Python-Lab-Work/week4/1.py"  
enter first number = 2 4  
(6, -2)  
PS E:\3rd sem\Python-Lab-Work> & "C:\Program Files\Python310\python.exe" "e:/3rd sem/Python-Lab-Work/week4/1.py"  
enter first number = 4 7  
(11, -3)  
PS E:\3rd sem\Python-Lab-Work> █
```

2. Write a function to return True if the first and last number of a given list are the same. If the numbers are different, return False.

Solution:

```
def check_list(lst):  
    return lst[0]==lst[-1]
```

```
l= list(map(int,input("enter a list of integer = ").split()))  
print(check_list(l))
```

```
PS E:\3rd sem\Python-Lab-Work> & "C:\Program Files\Python310\python.exe" "e:/3rd sem/Python-Lab-Work/week4/Q2.py"  
enter a list of integer = 3 4 5 6 7 5 6 7  
False  
PS E:\3rd sem\Python-Lab-Work> & "C:\Program Files\Python310\python.exe" "e:/3rd sem/Python-Lab-Work/week4/Q2.py"  
enter a list of integer = 2 3 4 5 6 2  
True  
PS E:\3rd sem\Python-Lab-Work> █
```

3. Given a list of numbers, write a program to turn every item of the list into its square.

Solution:

```
li= list(map(int,input("enter a list of integer = ").split()))  
print(list(map(lambda x:x**2,li)))
```

```

PS E:\3rd sem\Python-Lab-Work> & "C:\Program Files\Python310\python.exe" "e:/3rd sem/Python-Lab-Work/week4/Q
3.py"
enter a list of integer = 3 4 5 6 7 8 3 2
[9, 16, 25, 36, 49, 64, 9, 4]
PS E:\3rd sem\Python-Lab-Work> & "C:\Program Files\Python310\python.exe" "e:/3rd sem/Python-Lab-Work/week4/Q
3.py"
enter a list of integer = 3 4 5 6
[9, 16, 25, 36]
PS E:\3rd sem\Python-Lab-Work>

```

4. Given two Python lists, write a program to iterate both lists simultaneously and display items from list1 in original order and items from list 2 in reverse order.

Soulution:

```

from itertools import zip_longest
l1= [1,2,3,4,5]
l2=[1,2,3,4,5,6,7]
for a,b in zip_longest(l1,reversed(l2),fillvalue=0):
    print(a,b)

```

```

PS E:\3rd sem\Python-Lab-Work> & "C:\Program Files\Python310\python.exe" "e:/3rd sem/Python-Lab-Work/week4/Q
4.py"
1 7
2 6
3 5
4 4
5 3
0 2
0 1
PS E:\3rd sem\Python-Lab-Work>

```

5. Write a program to count the number of occurrences of item 50 in the tuple tp1 = (50, 10, 60, 70, 50).

Solution:

```

tp1=(50,10,60,70,50)
print("tuple = ",tp1)
count=tp1.count(50)
print("number of occurrences = ", count)

```

```
PS E:\3rd sem\Python-Lab-Work> & "C:\Program Files\Python310\python.exe" "e:/3rd sem/Python-Lab-Work/week4/Q
5.py"
tuple = (50, 10, 60, 70, 50)
number of occurence = 2
PS E:\3rd sem\Python-Lab-Work> \
```

Week #5

1. Write a program to generate a 6-digit random secure OTP.

Solution:

```
import secrets
otp= secrets.randbelow(900000)+100000
print(otp)
```



```
PS E:\3rd sem\Python-Lab-Work> & "C:\Program Files\Python310\python.exe" "e:/3rd sem/Python-Lab-Work/week5/Q
1.py"
708807
PS E:\3rd sem\Python-Lab-Work> & "C:\Program Files\Python310\python.exe" "e:/3rd sem/Python-Lab-Work/week5/Q
1.py"
114534
PS E:\3rd sem\Python-Lab-Work> & "C:\Program Files\Python310\python.exe" "e:/3rd sem/Python-Lab-Work/week5/Q
1.py"
842186
PS E:\3rd sem\Python-Lab-Work>
```

2. Write a program to pick a random character from a user-supplied string.

Solution:

```
import secrets
str= input("enter a string = ")
charac= secrets.choice(str)
print(charac)
```



```
enter a string = The world is beautiful
d
PS E:\3rd sem\Python-Lab-Work> & "C:\Program Files\Python310\python.exe" "e:/3rd sem/Python-Lab-Work/week5/Q
2.py"
enter a string = Hello
l
PS E:\3rd sem\Python-Lab-Work>
```

3. Write a program to generate a random password that meets the following conditions:
- Password length must be 10 characters long.
 - It must contain at least 2 uppercase letters, 1 digit, and 1 special symbol.

Solution:

```
import random
import string

uppercase = random.choices(string.ascii_uppercase, k=2)
digit = random.choice(string.digits)
special = random.choice("!@#$%^&*")

remaining = random.choices(
    string.ascii_letters + string.digits + "!@#$%^&*",
    k=6
)
password_list = uppercase + [digit] + [special] + remaining

random.shuffle(password_list)

password = ''.join(password_list)

print("Random Password:", password)
```



4. Given two lists of numbers, write a program to create a new list containing odd numbers from the first list and even numbers from the second list.

Solution:

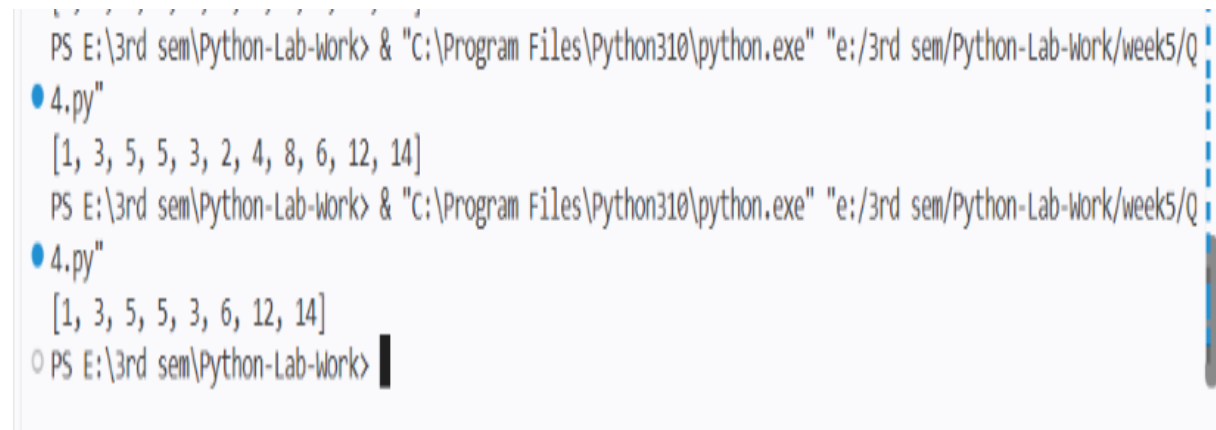
```
l1= [1,2,3,4,5,6,5,3]
l2= [2 ,3, 4,8, 7, 6, 5,12,14]
```

```

new_l=[]

for i in l1:
    if i%2!=0:
        new_l.append(i)
for i in l2:
    if i%2==0:
        new_l.append(i)
print(new_l)

```



```

PS E:\3rd sem\Python-Lab-Work> & "C:\Program Files\Python310\python.exe" "e:/3rd sem/Python-Lab-Work/week5/Q
4.py"
[1, 3, 5, 5, 3, 2, 4, 8, 6, 12, 14]
PS E:\3rd sem\Python-Lab-Work> & "C:\Program Files\Python310\python.exe" "e:/3rd sem/Python-Lab-Work/week5/Q
4.py"
[1, 3, 5, 5, 3, 6, 12, 14]
PS E:\3rd sem\Python-Lab-Work>

```

5. Write a program to create a numpy array and return an array of odd rows and even columns from the numpy array.

Solution:

```

import numpy as np
arr = np.array([
    [1, 2, 3, 4],
    [5, 6, 7, 8],
    [9, 10, 11, 12],
    [13, 14, 15, 16],
    [17, 18, 19, 20]
])
result = arr[::2, 1::2]
print("Original Array:")
print(arr)

print("Odd rows and even columns:")
print(result)

```

```

PS E:\3rd sem\Python-Lab-Work> & "C:\Program Files\Python310\python.exe" "e:/3rd sem/Python-Lab-Work/week5/Q
5.py"
Original Array:
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]
 [13 14 15 16]
 [17 18 19 20]]
Odd rows and even columns:
[[ 2  4]
 [10 12]
 [18 20]]

```

6. Write a program to create a numpy array and sort it as per the following cases:

- a. Case 1: Sort the array by the second row.
- b. Case 2: Sort the array by the second column.

Solution:

```

import numpy as np

arr = np.array([
    [30, 20, 10],
    [3, 1, 2],
    [9, 7, 8]
])

print("Original Array:")
print(arr)

# a. Case 1: Sort the array by the second row.

index = np.argsort(arr[1])

result = arr[:, index]

print("Sorted by second row:")
print(result)

# b. Case 2: Sort the array by the second column.
index = np.argsort(arr[:, 1])

result = arr[index]

print("Sorted by second column:")

```

```
print(result)
```

```
PS E:\3rd sem\Python-Lab-Work> & "C:\Program Files\Python310\python.exe" "e:/3rd sem/Python-Lab-Work/week5/Q
6.py"
● Original Array:
[[30 20 10]
 [ 3  1  2]
 [ 9  7  8]]
Sorted by second row:
[[20 10 30]
 [ 1  2  3]
 [ 7  8  9]]
Sorted by second column:
[[ 3  1  2]
 [ 9  7  8]
 [30 20 10]]
○ PS E:\3rd sem\Python-Lab-Work> █
```

Week #6

1. Write a Python program that inputs two tuples and creates a third that contains all elements of the first followed by all elements of the second.

Solution

```
tup1= tuple(map(int,input("enter numbers for tuple1 = ").split()))
tup2=tuple(map(int,input("enter numbers for tuple2 = ").split()))

tup3= tup1 + tup2
print("contains all element from tup1 to tup2 = ", tup3)
sorted_tup3= sorted(tup3)
print("sorted tuple = ", sorted_tup3)
```

```
PS E:\3rd sem\Python-Lab-Work> & "C:\Program Files\Python310\python.exe" "e:/3rd sem/Python-Lab-Work/week6/Q1.py"
● enter numbers for tuple1 = 2 3 4 5 6
  enter numbers for tuple2 = 3 4 5 6 9 8
  contains all element from tup1 to tup2 = (2, 3, 4, 5, 6, 3, 4, 5, 6, 9, 8)
  sorted tuple = [2, 3, 3, 4, 4, 5, 5, 6, 6, 8, 9]
○ PS E:\3rd sem\Python-Lab-Work> █
```

2. Write a Python program to create a dictionary with names and phone numbers. Then ask the user for a name and print the corresponding phone number.

Solution

```
user={
    "Sumaniya":4746375862,
    "Hadiya":8766375862,
    "Rashika":3986375843,}
name=input("input name = ")
for key in user:
    if key.casefold()==name.casefold():
        print(user[key])
```

```
Problems 5 Output Debug Console Terminal Ports
● PS E:\3rd sem\Python-Lab-Work> & "C:\Program Files\Python310\python.exe" "e:/3rd sem/Python-Lab-Work/week6/Q2.py"
  input name = sumaniya
  4746375862
● PS E:\3rd sem\Python-Lab-Work> & "C:\Program Files\Python310\python.exe" "e:/3rd sem/Python-Lab-Work/week6/Q2.py"
  input name = Sumaniya
  4746375862
○ PS E:\3rd sem\Python-Lab-Work> █
```

3. Write a Python program to calculate the sum of squares of the first two digits and the last two digits of a 4-digit number, e.g., for 1233, calculate $12^2 + 33^2$.

Solution

```
num= (input("enter a 4 digit number = "))
sum_square= (lambda x,y: int(x)**2 + int(y)**2)(num[0:2], num[2:])
print(sum_square)
```

```
PS E:\3rd sem\Python-Lab-Work> & "C:\Program Files\Python310\python.exe" "e:/3rd sem/Python-Lab-Work/week6/Q3.py"
enter a 4 digit number = 3456
4292
PS E:\3rd sem\Python-Lab-Work> & "C:\Program Files\Python310\python.exe" "e:/3rd sem/Python-Lab-Work/week6/Q3.py"
enter a 4 digit number = 1212
288
PS E:\3rd sem\Python-Lab-Work>
```

4. Write a program that inputs a main string and creates an encrypted string by embedding a short symbol-based string after each character. The program should also decrypt the string.

Solution

```
# str1= input("Enter the first string: ")
# str2=""
# str3=""
# for i in range(len(str1)):
#     str2=str2+str1[i]+"@#"
# print("encrypted : ",str2)

# for i in range(len(str2)):
#     if str2[i]!="@" and str2[i]!="#":
#         str3=str3+str2[i]
# print("decrypted : ",str3)

str1 = input("Enter the first string: ")

str2 = "@#".join(str1) + "@#"
print("encrypted : ", str2)

str3 = str2.replace("@#", "")
print("decrypted : ", str3)
```

```
Problems Output Debug Console Terminal Ports
Python + - [ ] [X] ... [ ] [X]

PS E:\3rd sem\Python-Lab-Work> & "C:\Program Files\Python310\python.exe" "e:/3rd sem/Python-Lab-Work/week6/Q4.py"
Enter the first string: hello world
encrypted : h@#e@#l@#l@#o@# @#w@#o@#r@#l@#d@#
decrypted : hello world
PS E:\3rd sem\Python-Lab-Work>
```

5. Write a program to get roll numbers, names, and marks of students and store these details in a file called "Marks.data".

Solution

```
f= open("marks.data", "a")
# f.write("Name\t\tRoll\t\tMarks\n")
num_std =int(input("Enter number of students: "))

for i in range(num_std):
    name=input("Enter the name of student = ")
    roll_no = input("enter the roll number of student = ")
    marks = input("enter the marks of student = ")
    f.write(name+"\t\t"+roll_no+"\t\t"+marks+"\n")
f.close()

f= open("marks.data", "r")
data=f.read()
print(data)
f.close()
```

```
Problems Output Debug Console Terminal Ports
Python + - [ ] [X] ... [ ] [X]

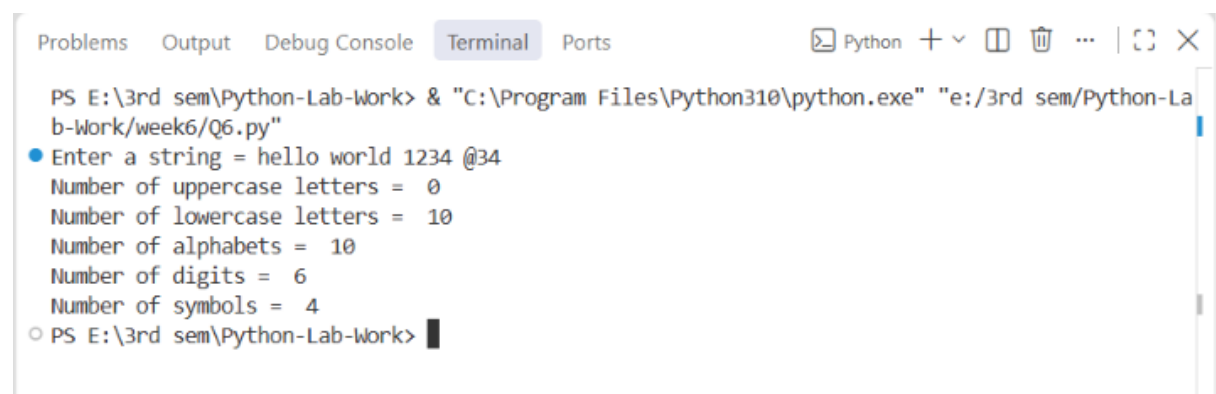
PS E:\3rd sem\Python-Lab-Work> & "C:\Program Files\Python310\python.exe" "e:/3rd sem/Python-Lab-Work/week6/Q5.py"
Name          Roll          Marks
Sumaniya      25CAMSA144    90
hadiya        25CAMSA132    90
rashika       25CAMSA133    90
iqra          25CAMSA116    90
faiza         25CAMSA120    90
PS E:\3rd sem\Python-Lab-Work>
```

6. Write a program to accept a string and display:
- Number of uppercase characters.
 - Number of lowercase characters.

- c. Total number of alphabets.
- d. Number of digits.

Solution

```
string = input("Enter a string = ")
uppercase = 0
lowercase = 0
alphabets = 0
digits = 0
symbols = 0
for i in string:
    if i.isupper():
        uppercase += 1
    if i.islower():
        lowercase += 1
    if i.isalpha():
        alphabets += 1
    if i.isdigit():
        digits += 1
    if not i.isalpha() and not i.isdigit():
        symbols += 1
print("Number of uppercase letters = ", uppercase)
print("Number of lowercase letters = ", lowercase)
print("Number of alphabets = ", alphabets)
print("Number of digits = ", digits)
print("Number of symbols = ", symbols)
```



```
PS E:\3rd sem\Python-Lab-Work> & "C:\Program Files\Python310\python.exe" "e:/3rd sem/Python-Lab-Work/week6/Q6.py"
Enter a string = hello world 1234 @34
Number of uppercase letters = 0
Number of lowercase letters = 10
Number of alphabets = 10
Number of digits = 6
Number of symbols = 4
PS E:\3rd sem\Python-Lab-Work>
```

7. Create a C program that accepts an IPv4 address and subnet mask in dotted decimal format, and outputs:
- a. Network ID
 - b. Broadcast Address
 - c. First and Last Host Address
 - d. CIDR notation
 - e. Number of valid hosts

f. IP Address Class (A/B/C/D/E)

Solution

```
#include <stdio.h>
#include <stdint.h>

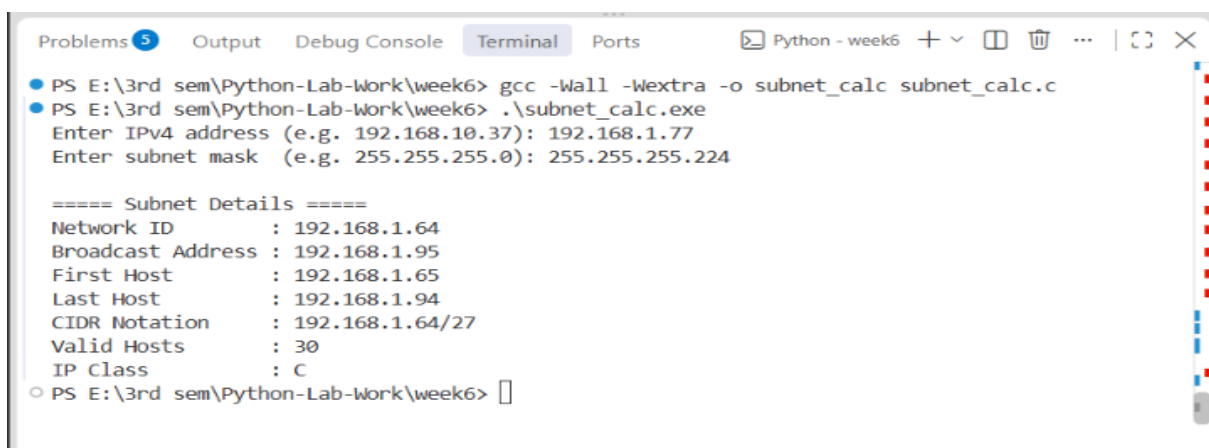
static void show(const char *label, uint32_t x)
{
    printf("%-18s: %u.%u.%u.%u\n", label, x >> 24, (x >> 16) & 255, (x >> 8) & 255,
x & 255);
}

int main(void)
{
    unsigned a, b, c, d, m, n, o, p;

    printf("IP address : "); scanf("%u.%u.%u.%u", &a, &b, &c, &d);
    printf("Subnet mask: "); scanf("%u.%u.%u.%u", &m, &n, &o, &p);

    uint32_t ip   = a << 24 | b << 16 | c << 8 | d;
    uint32_t mask = m << 24 | n << 16 | o << 8 | p;
    uint32_t net  = ip & mask, bc = net | ~mask;
    int cidr = __builtin_popcount(mask); /* count the 1 bits */
    unsigned long long hosts = cidr >= 31 ? 33ULL - cidr : (1ULL << (32 - cidr)) -
    2;
    show("Network ID", net);
    show("Broadcast Address", bc);
    show("First Host", cidr < 31 ? net + 1 : net);
    show("Last Host", cidr < 31 ? bc - 1 : bc);
    printf("%-18s: %d\n", "CIDR Notation", cidr);
    printf("%-18s: %llu\n", "Valid Hosts", hosts);
    printf("%-18s: %c\n", "IP Class", a < 128 ? 'A' : a < 192 ? 'B' : a < 224 ? 'C'
: a < 240 ? 'D' : 'E');

    return 0;
}
```



The screenshot shows a terminal window with the following content:

```
Problems 5 Output Debug Console Terminal Ports Python - week6 + - [ ] [X]
• PS E:\3rd sem\Python-Lab-Work\week6> gcc -Wall -Wextra -o subnet_calc subnet_calc.c
• PS E:\3rd sem\Python-Lab-Work\week6> .\subnet_calc.exe
Enter IPv4 address (e.g. 192.168.10.37): 192.168.1.77
Enter subnet mask (e.g. 255.255.255.0): 255.255.255.224

===== Subnet Details =====
Network ID       : 192.168.1.64
Broadcast Address : 192.168.1.95
First Host       : 192.168.1.65
Last Host        : 192.168.1.94
CIDR Notation    : 192.168.1.64/27
Valid Hosts      : 30
IP Class         : C
○ PS E:\3rd sem\Python-Lab-Work\week6> [ ]
```

Week #7

1. Write a report on different types of Linux shells (sh, csh, ksh, bash).

Aim:

To study different types of Linux shells: sh, csh, ksh, and bash.

Theory:

sh (Bourne Shell): The original Unix shell used for executing commands and shell scripts.

csh (C Shell): A shell with syntax similar to the C programming language.

ksh (Korn Shell): An advanced shell based on sh, supporting scripting and command history.

bash (Bourne Again Shell): An improved version of sh and one of the commonly used shells in Linux.

Result:

Different types of Linux shells were studied.

2. Copy, move, and remove files using cp, mv, and rm commands.

Aim:

To copy, move, and remove files using cp, mv, and rm commands.

Commands:

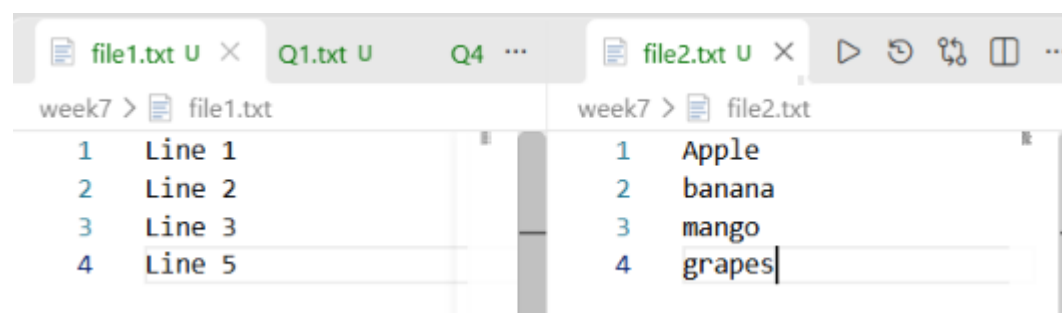
cp file1.txt file2.txt

mv file1.txt file2.txt

rm file1.txt

Result:

Files were successfully copied, moved, and removed using cp, mv, and rm commands.



```
week7 > file1.txt
1 Line 1
2 Line 2
3 Line 3
4 Line 5

week7 > file2.txt
1 Line 1
2 Line 2
3 Line 3
4 Line 5

PS E:\3rd sem\Python-Lab-Work\week7> cp file1.txt file2.txt
PS E:\3rd sem\Python-Lab-Work\week7> mv file2.txt test

week7 > file1.txt
1 Line 1
2 Line 2
3 Line 3
4 Line 5

week7 > test
1 Apple
2 banana
3 mango
4 grapes

PS E:\3rd sem\Python-Lab-Work\week7> mv file2.txt test
```

3. Create and delete directories using mkdir and rmdir.

mkdir = **make** folder
rmdir = **remove** folder

```
PS E:\3rd sem\Python-Lab-Work\week7> mkdir myfolder

Directory: E:\3rd sem\Python-Lab-Work\week7

Mode                LastWriteTime         Length Name
----                -
d-----          30-09-2026  01:40 AM                myfolder

PS E:\3rd sem\Python-Lab-Work\week7> rmdir myfolder
PS E:\3rd sem\Python-Lab-Work\week7>
```

4. Change the current working directory using `cd` and display the present working directory using `pwd`.

Aim:

To change the current working directory and display the present working directory using `cd` and `pwd` commands.

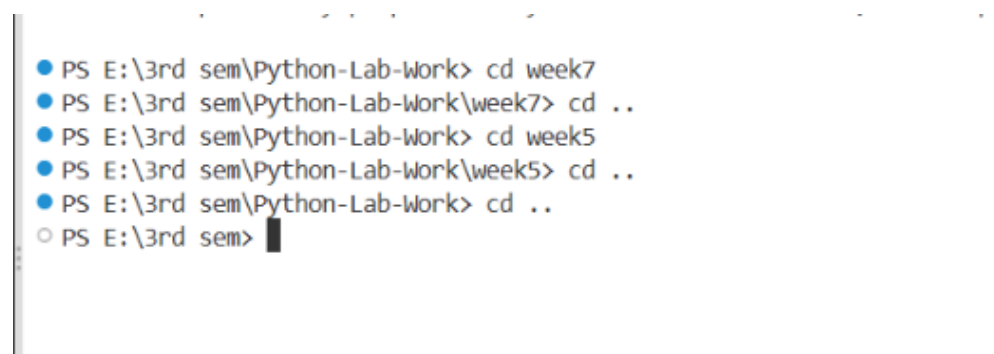
Commands:

```
pwd
cd Documents
pwd
cd ..
```

Result:

The current working directory was changed and displayed successfully using `cd` and `pwd` commands.

Then paste your **terminal screenshot** below the Result.



```
PS E:\3rd sem\Python-Lab-Work> cd week7
PS E:\3rd sem\Python-Lab-Work\week7> cd ..
PS E:\3rd sem\Python-Lab-Work> cd week5
PS E:\3rd sem\Python-Lab-Work\week5> cd ..
PS E:\3rd sem\Python-Lab-Work> cd ..
PS E:\3rd sem>
```

5. Consider two files that contain information about Employees and Departments in the following parameters: Employee (Name, EId, Salary, DID), Department (DID, DName, DLocation). Write a Python program to find the average salary of each department.

Solution

```
employee_file = open("Employee.txt", "r")

employees = employee_file.readlines()
employee_file.close()

salary = {}
```

```

for line in employees[1:]:
    data = line.strip().split(",")

    name = data[0]
    eid = data[1]
    sal = float(data[2])
    did = data[3]

    if did not in salary:
        salary[did] = []

    salary[did].append(sal)

for did in salary:
    average = sum(salary[did]) / len(salary[did])
    print(did, "Average Salary =", average)

```



```

Problems 2 Output Debug Console Terminal Ports Python - week7
• PS E:\3rd sem\Python-Lab-Work> cd week7
• PS E:\3rd sem\Python-Lab-Work\week7> & "C:\Program Files\Python310\python.exe" "e:/3rd
  sem/Python-Lab-Work/week7/Q5.py"
  D01 Average Salary = 35000.0
  D02 Average Salary = 55000.0
○ PS E:\3rd sem\Python-Lab-Work\week7>

```

6. Consider two files having some lines of statements. Write a Python program to swap content present at middle line of first file with the content of last line of the second file. (Note: First file contains odd numbers of lines of statement)

Solution

```

# Swap middle line of first file with last line of second file
f1 = open("file1.txt", "r")
lines1 = f1.readlines()
f1.close()

f2 = open("file2.txt", "r")
lines2 = f2.readlines()
f2.close()

middle = len(lines1) // 2

temp = lines1[middle]
lines1[middle] = lines2[-1]
lines2[-1] = temp

```

```

f1 = open("file1.txt", "w")
f1.writelines(lines1)
f1.close()

f2 = open("file2.txt", "w")
f2.writelines(lines2)
f2.close()

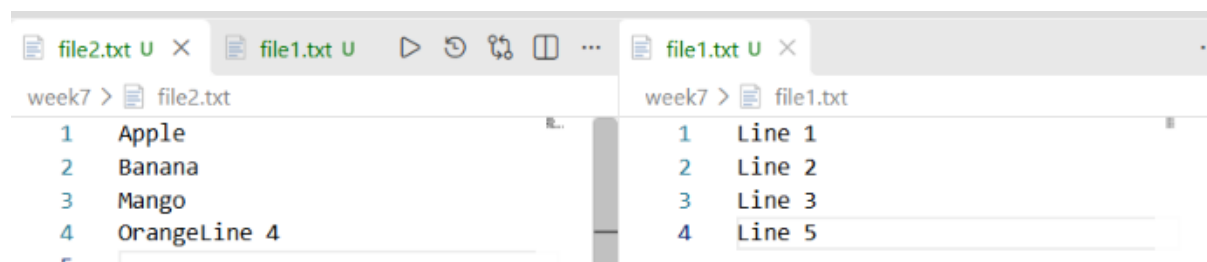
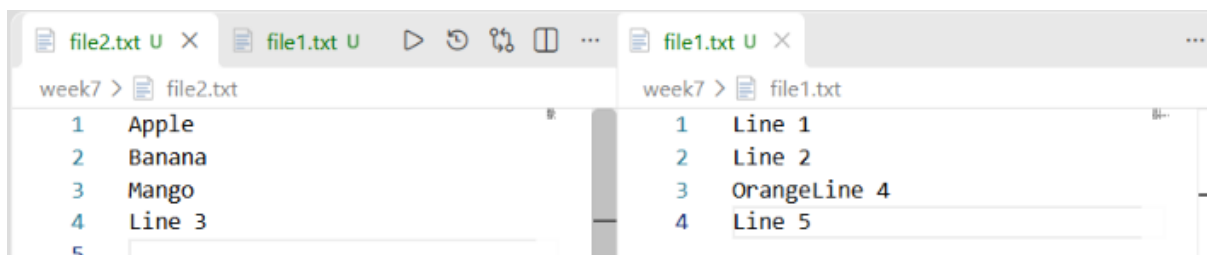
print("Content swapped successfully")

```

```

PS E:\3rd sem\Python-Lab-Work\week7> python Q6.py
Content swapped successfully
PS E:\3rd sem\Python-Lab-Work\week7>

```



7. Simulate a RIP-like behavior using a Python program. Accept a list of directly connected routers with hop counts and simulate RIP updates for 3 rounds. Show how the routing table evolves.

Solution

```

// Simulate RIP-like behavior for 3 rounds
// command to run
// compile gcc Q7.c -o Q7
// run .\Q7.exe
#include <stdio.h>

```

```

int main()
{
    int a[3][3] = {
        {0, 1, 3},
        {1, 0, 999},
        {3, 999, 0}
    };

    int i, j, k, round;

    for(round = 1; round <= 3; round++)
    {
        for(i = 0; i < 3; i++)
        {
            for(j = 0; j < 3; j++)
            {
                for(k = 0; k < 3; k++)
                {
                    if(a[i][k] + a[k][j] < a[i][j])
                    {
                        a[i][j] = a[i][k] + a[k][j];
                    }
                }
            }
        }

        printf("\nRound %d\n", round);

        for(i = 0; i < 3; i++)
        {
            for(j = 0; j < 3; j++)
            {
                if(a[i][j] == 999)
                    printf("INF ");
                else
                    printf("%d ", a[i][j]);
            }

            printf("\n");
        }
    }

    return 0;
}

```

```
Problems 2 Output Debug Console Terminal Ports Python - week7 + - [ ] [X] ... [ ] [X] X
● PS E:\3rd sem\Python-Lab-Work\week7> .\Q7.exe

Round 1
0 1 3
1 0 4
3 4 0

Round 2
0 1 3
1 0 4
3 4 0

Round 3
0 1 3
1 0 4
3 4 0
○ PS E:\3rd sem\Python-Lab-Work\week7> █
```